

Recurrence Relations Explained



teach me recurrence relation fully in details

Absolutely! Let's dive deep into **Recurrence Relations**, which are foundational in algorithms, mathematics, and computer science — especially in **complexity analysis** of divide-and-conquer algorithms.

◆ What is a Recurrence Relation?

A **recurrence relation** is an equation that defines a sequence recursively: each term is defined as a function of its preceding terms.

For example, the **Fibonacci sequence**:

$$F(n) = F(n - 1) + F(n - 2), \quad \text{with } F(0) = 0, F(1) = 1$$

In algorithms, it often arises when analyzing the **time complexity** of recursive functions.

◆ Why is it important?

Suppose an algorithm breaks a problem of size n into subproblems and combines the results. Its runtime is expressed as a recurrence. For example:

Merge Sort:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

We want to **solve** this recurrence to find the time complexity — $T(n)$.

◆ Types of Recurrence Relations

1. Linear Recurrence with Constant Coefficients

Example:

$$T(n) = 2T(n - 1) + 3$$

Each term depends on a fixed number of previous terms.

2. Divide-and-Conquer Recurrence

Example:

$$T(n) = 2T(n/2) + n$$

Typical in recursive algorithms like Merge Sort, Binary Search, etc.

◆ Solving Recurrence Relations

There are **four main methods**:

✓ 1. Substitution Method (a.k.a. Guess and Check)

Steps:

1. Guess the solution.
2. Use **mathematical induction** to prove it.

Example:

$$T(n) = 2T(n/2) + n \quad \text{with } T(1) = 1$$

Step 1: Guess

Let's guess: $T(n) = O(n \log n)$

Step 2: Prove by induction

Assume for all $k < n$,

$$T(k) \leq ck \log k$$

Now,

$$\begin{aligned} T(n) &= 2T(n/2) + n \leq 2 \cdot c \cdot \frac{n}{2} \log \left(\frac{n}{2} \right) + n \\ &= cn \log \left(\frac{n}{2} \right) + n = cn(\log n - 1) + n = cn \log n - cn + n \end{aligned}$$

So, for a large enough c , we get:

$$T(n) \leq cn \log n$$

● **Proved**

✓ 2. Recursion Tree Method

Steps:

- Draw a tree of recursive calls
- Calculate work done at each level
- Sum up all the levels

Example:

$$T(n) = 2T(n/2) + n$$

Tree:

- Level 0: n
- Level 1: $2 \cdot (n/2) = n$
- Level 2: $4 \cdot (n/4) = n$
- ...
- Level $\log n$: total work = n

There are $\log n + 1$ levels, each doing n work

→ Total work = $n \log n$

● So, $T(n) = O(n \log n)$

✓ 3. Master Theorem

Works for recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

Where:

- $a \geq 1, b > 1$
- $f(n)$ is an asymptotic function (like $n, n \log n, n^2$, etc.)

Master Theorem Cases:

Let $f(n) \in \Theta(n^d)$

1. If $d < \log_b a \rightarrow$

$$T(n) = \Theta(n^{\log_b a})$$

2. If $d = \log_b a \rightarrow$

$$T(n) = \Theta(n^d \log n)$$

3. If $d > \log_b a$ and regularity condition holds $\rightarrow$

$$T(n) = \Theta(f(n))$$

Examples:

♦ $T(n) = 2T(n/2) + n$

- $a = 2, b = 2, f(n) = n$
- $\log_b a = \log_2 2 = 1$, and $f(n) = n = n^1$
- ✓ Case 2 $\rightarrow T(n) = \Theta(n \log n)$

- ♦ $T(n) = 4T(n/2) + n^2$
 - $a = 4, b = 2 \Rightarrow \log_2 4 = 2$
 - $f(n) = n^2 = n^d, d = 2$
 - ✓ Case 2 again $\rightarrow \Theta(n^2 \log n)$
- ♦ $T(n) = 2T(n/2) + n^2$
 - $\log_2 2 = 1$, but $d = 2 > 1$
 - ✓ Case 3 $\rightarrow \Theta(n^2)$

✓ 4. Iteration / Expansion Method

Steps:

- Expand the recurrence multiple times
- Find a pattern
- Simplify it

Example:

$$T(n) = 2T(n/2) + n$$

Expand:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n \\ &= 8T(n/8) + 3n \\ &= \dots = 2^k T(n/2^k) + kn \end{aligned}$$

Let $n/2^k = 1 \Rightarrow k = \log n$

$$T(n) = nT(1) + n \log n = \Theta(n \log n)$$

◆ Common Recurrence Forms (Cheat Sheet)

Recurrence	Time Complexity
$T(n) = T(n-1) + c$	$O(n)$
$T(n) = 2T(n/2) + n$	$O(n \log n)$
$T(n) = T(n/2) + c$	$O(\log n)$
$T(n) = 2T(n/2) + n^2$	$O(n^2)$
$T(n) = 4T(n/2) + n$	$O(n^2)$

◆ Practice Problems

1. $T(n) = T(n/2) + \log n$
2. $T(n) = 3T(n/4) + n$
3. $T(n) = 8T(n/2) + n^2$
4. $T(n) = 2T(n/2) + n/\log n$

Want me to help you solve these? Or would you like PDF/YouTube recommendations to master them too?



explain why merge sort is $T(n)=2T(\frac{n}{2})+O(n)$

Great question!

Let's break down **why the recurrence for Merge Sort is:**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

◆ What is Merge Sort?

Merge Sort is a **divide-and-conquer** algorithm. It works in 3 main steps:

1. **Divide:** Split the array into two halves.
2. **Conquer:** Recursively sort each half.
3. **Combine:** Merge the two sorted halves into one sorted array.

◆ Understanding the Recurrence

Let's analyze the **time complexity** step by step:

✓ Step 1: Divide

- Splitting the array into two halves takes **$O(1)$** time.
(Just calculate $\text{mid} = (\text{low} + \text{high})/2$)

✓ So this part is **negligible**.

✓ Step 2: Conquer

- You recursively call `mergeSort()` on the **two halves** of size $n/2$.

So, this gives:

$$2T\left(\frac{n}{2}\right)$$

✓ Step 3: Combine (Merge Step)

- Merging two **sorted arrays** of total size n takes **$O(n)$** time.

How?

You use two pointers — one for each half — and go through each element once, comparing and merging:

```
cpp
while (i < n1 && j < n2) {
    if (L[i] <= R[j])
        A[k++] = L[i++];
    else
        A[k++] = R[j++];
}
```

→ This loop runs for **all n elements once**.

So the merge step is:

$$O(n)$$

↻ Combine All Steps

So, the total time for an input size of n is:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

◆ What does this recurrence mean?

Each time we:

- Split** the array into two equal halves,
- Recursively sort** each half (which is $T(n/2)$),
- Merge** the two sorted halves (which is $O(n)$).

So the work at each level is:

Level	# of calls	Size of each subproblem	Total work
0	1	n	n
1	2	$n/2$	$2 \cdot n/2 = n$
2	4	$n/4$	$4 \cdot n/4 = n$

Level	# of calls	Size of each subproblem	Total work
...	...	...	n
$\log_2 n$	n calls	size = 1	work = n

✓ At every level, we do $O(n)$ work.

✓ There are $\log_2(n)$ levels (because we divide the size by 2 each time).

◆ Final Time Complexity

$$T(n) = O(n \log n)$$

✓ So, Merge Sort runs in $O(n \log n)$ time.

◆ Summary

Step	Time
Divide	$O(1)$
Conquer	$2T(n/2)$
Merge	$O(n)$
Total	$T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$

Let me know if you'd like to see this solved using **Recursion Tree**, **Master Theorem**, or **Substitution** for better clarity.